TF-IDF EXPLAINED

Turn text into numbers that capture **importance**.

TF-IDF = TF × IDF

Term Frequency × Inverse Document Frequency

THE IDEA

TF
How often a word appears in a document

×

IDF
How rare a word is across all documents

TF-IDF
Important in this document BUT rare across all documents

WHAT IS TF-IDF?

A statistical measure that evaluates how important a word is in a document relative to a collection of documents.

Used in:

- Search Engines
- Recommendation Systems
- Spam Filtering
- Chatbots
- Information Retrieval
- Text Classification

OUR EXAMPLE CORPUS

3 Documents (Sentences)

S_1: good boy

S_2: good girl

S_3: boy girl good

Vocabulary (unique words)

good | boy | girl

STEP 1 TERM FREQUENCY (TF)

Frequency of a word in a document divided by total words in that document.

$$TF(t, d) = \frac{\text{count of term } t \text{ in document } d}{\text{total terms in document } d}$$

TF TABLE

Word	S_1 (good boy) (2 words)	S_2 (good girl) (2 words)	S_3 (boy girl good) (3 words)
good	1/2	1/2	1/3
boy	1/2	0	1/3
girl	0	1/2	1/3

How to read:
In S_1 ("good boy"), "good" appears 1 time out of 2 total words → TF = 1/2

STEP 2 INVERSE DOCUMENT FREQUENCY (IDF)

Measures how rare a word is across all documents. Common words get lower importance.

$$IDF(t) = \log\left(\frac{N}{df(t)}\right)$$

Where:

N = total number of documents

$df(t)$ = number of documents containing term t

STEP 3 TF-IDF (TF × IDF)

Multiply TF and IDF to get the final importance score.

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

FINAL TF-IDF WEIGHT MATRIX

Word	S_1 (good boy)	S_2 (good girl)	S_3 (boy girl good)
good	0	0	0
boy	$\frac{1}{2}\log(3/2)$	0	$\frac{1}{3}\log(3/2)$
girl	0	$\frac{1}{2}\log(3/2)$	$\frac{1}{3}\log(3/2)$

Vector Representation
(ordered as [good, boy, girl])

$S_1 = [\ 0,\ \frac{1}{2}\log(3/2),\ 0\ ]$

$S_2 = [\ 0,\ 0,\ \frac{1}{2}\log(3/2)\ ]$

$S_3 = [\ 0,\ \frac{1}{3}\log(3/2),\ \frac{1}{3}\log(3/2)\ ]$

ADVANTAGES

- Intuitive and easy to understand
- Captures word importance
- Fixed-size numerical representation
- Works well for many NLP tasks
- Simple and efficient

DISADVANTAGES

- Produces sparse vectors
- Out of Vocabulary (OOV) problem
- Ignores word order (Bag-of-Words model)
- No semantic / contextual meaning
- King and queen are unrelated mathematically

KEY TAKEAWAY

TF-IDF scores are high when a word is important in a document but rare across all documents.

Common words like "good" get downweighted. Unique words like "boy" and "girl" stand out.

FYI

Modern systems may use a smoothed IDF to avoid zero values:

$$IDF(t) = \log\left(\frac{1+N}{1+df(t)}\right) + 1$$

REAL-WORLD USE CASES

Search Engines | Document Ranking | Spam Detection | Recommendation Systems | Chatbots & NLP Pipelines

IN SHORT TF captures local importance, IDF captures global rarity. TF-IDF captures what truly matters.

```
import pandas as pd
messages = pd.read_csv('SMSSpamCollection.txt', sep='\t',
                           names=["label", "message"])
messages
```

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...
...	...	...
5567	spam	This is the 2nd time we have tried 2 contact u...
5568	ham	Will ü b going to esplanade fr home?
5569	ham	Pity, * was in mood for that. So...any other s...
5570	ham	The guy did some bitching but I acted like i'd...
5571	ham	Rofl. Its true to its name

5572 rows × 2 columns

```
## Data Cleaning and preprocessing
import re
import nltk
nltk.download('stopwords')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
True
```

```
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
lem  = WordNetLemmatizer()
```

```
corpus = []

for i in range(0, len(messages)):
    review = re.sub('[^a-zA-Z]',' ',messages['message'][i])
    review = review.lower()
    review = review.split()
    review = [lem.lemmatize(word) for word in review if not word in stopwords.words("english")]
    review = ' '.join(review)
    corpus.append(review)
```

```
corpus
```

```
 willing go aptitude class',
 'wont b trying sort house ok',
 'yar lor wan go c horse racing today mah eat earlier lor ate chicken rice u',
 'haha awesome omw back',
 'yup thk e shop close lor',
 'account number',
 'eh u send wrongly lar',
 'hey ad crap nite borin without ya boggy u boring biatch thanx u wait til nxt time il ave ya',
 'ok shall talk',
 'dont hesitate know second time weakness like keep notebook eat day anything changed day sure nothing',
 'hey pay salary de lt gt',
 'another month need chocolate weed alcohol',
 'started searching get job day great potential talent',
 'reckon need town eightish walk carpark',
 'congrats mobile g videophones r call videochat wid mate play java game dload polyph music noline rentl',
 'look fuckin time fuck think',
 'yo guess dropped',
 'carlos say mu lt gt minute',
 'office call lt gt min',
 'geeee miss already know think fuck wait till next year together loving kiss',
 'yun ah ubi one say wan call tomorrow call look irene ere got bus ubi cres ubi tech park ph st wkg day n',
 'ugh gotta drive back sd la butt sore',
 'th july',
 'hi im relaxing time ever get every day party good night get home tomorrow ish',
 'wan come come lor din c stripe skirt',
 'xmas story peace xmas msg love xmas miracle jesus hav blessed month ahead amp wish u merry xmas',
 'number',
 'change e one next escalator',
 'yetunde class run water make ok pls',
 'lot happened feel quiet beth aunt charlie working lot helen mo',
 'wait bus stop aft ur lect lar dun c go get car come back n pick',
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
tfidf = TfidfVectorizer(max_features=100)
X = tfidf.fit_transform(corpus).toarray()
```

```
import numpy as np
np.set_printoptions(edgeitems=30, linewidth=100000,
  formatter=dict(float=lambda x: "%.3g" % x))
```

x

```
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ..., 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0.287, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.364, 0, 0.321, 0, 0, 0, 0, 0, 0.431, 0, 0, ..., 0, 0, 0, 0, 0,
0, 0, 0.378, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0.369, 0, 0, 0.555, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ..., 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0.469, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.706, 0, 0, 0, 0, 0, ..., 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ..., 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.548, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ..., 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.631, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ..., 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]])
```

```python
# with N-Grams

tfidf = TfidfVectorizer(max_features=100,ngram_range=(2,2))
X = tfidf.fit_transform(corpus).toarray()
```

```python
tfidf.vocabulary_
```

```
 'good night': np.int64(38),
```

```
 'network min': np.int64(62),
 'camcorder reply': np.int64(12),
 'reply call': np.int64(75),
 'last night': np.int64(53),
 'camera phone': np.int64(13),
 'pick phone': np.int64(67),
 'pls send': np.int64(69),
 'send message': np.int64(77),
 'great day': np.int64(39),
 'ur cash': np.int64(91),
 'suite land': np.int64(83),
 'land row': np.int64(52),
 'good afternoon': np.int64(36),
 'take care': np.int64(84),
 'double min': np.int64(28),
 'call mobileupd': np.int64(9),
 'call optout': np.int64(10),
 'gt min': np.int64(41),
 'txt stop': np.int64(88),
 'dating service': np.int64(25),
 'pobox wq': np.int64(71),
 'mobile number': np.int64(59),
 'call land': np.int64(6),
 'land line': np.int64(51),
 'line claim': np.int64(56),
 'claim valid': np.int64(19),
 'gt lt': np.int64(40),
 'hope good': np.int64(49),
 'free text': np.int64(34),
 'holiday cash': np.int64(48),
 'prize claim': np.int64(73),
 'nd attempt': np.int64(61),
 'attempt contact': np.int64(1),
 'claim ur': np.int64(18),
 'ok lor': np.int64(66),
 'want come': np.int64(96),
```

Start coding or generate with AI.

Start coding or generate with AI.